

Exercise 2: E-commerce Platform Search Function

Understanding Asymptotic Notation:

Big O Notation is used to describe the efficiency of an algorithm by measuring how its execution time grows as the input size increases.

Common Big O complexities:

Complexity	Name	Example
$O(1)$	Constant	Accessing an array element
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort

Search Operation Scenarios

Linear Search

- **Best Case:** $O(1)$
 - Target found at first position.
- **Average Case:** $O(n/2) \approx O(n)$
 - Target found somewhere in the middle.
- **Worst Case:** $O(n)$
 - Target is last element or not present.

Binary Search

- **Best Case:** $O(1)$
 - Target found at middle element.
- **Average Case:** $O(\log n)$

- **Worst Case:** $O(\log n)$
 - Search space repeatedly divided until one element remains.

Product.java

```
public class Product {  
  
    private int productId;  
  
    private String productName;  
  
    private String category;  
  
  
    public Product(int productId, String productName, String category) {  
  
        this.productId = productId;  
  
        this.productName = productName;  
  
        this.category = category;  
  
    }  
  
  
    public int getProductId() {  
  
        return productId;  
  
    }  
  
  
    public String getProductName() {  
  
        return productName;  
  
    }  
  
  
    public String getCategory() {  
  
        return category;  
  
    }  
}
```

```
@Override  
public String toString() {  
    return "Product ID: " + productId +  
        ", Name: " + productName +  
        ", Category: " + category;  
}  
}
```

LinearSearch.java

```
public class LinearSearch {  
  
    public static Product search(Product[] products, int targetId) {  
  
        for (Product product : products) {  
            if (product.getId() == targetId) {  
                return product;  
            }  
        }  
  
        return null;  
    }  
}
```

BinarySearch.java

```
public class BinarySearch {  
  
    public static Product search(Product[] products, int targetId) {
```

```
int left = 0;

int right = products.length - 1;

while (left <= right) {

    int mid = left + (right - left) / 2;

    if (products[mid].getProductId() == targetId) {
        return products[mid];
    }

    if (products[mid].getProductId() < targetId) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return null;
}
```

EcommerceSearchDemo.java

```
public class EcommerceSearchDemo {

    public static void main(String[] args) {
```

```
Product[] products = {  
    new Product(101, "Laptop", "Electronics"),  
    new Product(102, "Smartphone", "Electronics"),  
    new Product(103, "Shoes", "Fashion"),  
    new Product(104, "Watch", "Accessories"),  
    new Product(105, "Headphones", "Electronics")  
};
```

```
int targetId = 104;
```

```
// Linear Search
```

```
Product linearResult =
```

```
    LinearSearch.search(products, targetId);
```

```
if (linearResult != null) {
```

```
    System.out.println("Linear Search Found:");
```

```
    System.out.println(linearResult);
```

```
} else {
```

```
    System.out.println("Product not found.");
```

```
}
```

```
// Binary Search
```

```
Product binaryResult =
```

```
    BinarySearch.search(products, targetId);
```

```
if (binaryResult != null) {
```

```

        System.out.println("\nBinary Search Found:");

        System.out.println(binaryResult);

    } else {

        System.out.println("Product not found.");

    }

}

}

```

Output:

```

● PS D:\SD\Digital-Nurture-JavaFSE-main\local_repo\Cognizant-ex-1.0.6.7-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInException
ms\Exercise-2\E-Commerce Search\bin' 'EcommerceSearchDemo'
Linear Search Found:
Product ID: 104, Name: Watch, Category: Accessories

Binary Search Found:
Product ID: 104, Name: Watch, Category: Accessories

```

Analysis:

Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Which Algorithm is Better for an E-commerce Platform?

Linear Search:

Advantages

- Simple to implement.
- Works on unsorted data.

Disadvantages

- Slow for large datasets.
- Time increases linearly with number of products.

Binary Search:

Advantages

- Much faster for large product catalogs.
- Reduces search space by half in every iteration.

Disadvantages

- Requires sorted data.
- Maintaining sorted order may require extra processing.

Conclusion:

For a large e-commerce platform containing thousands or millions of products, **Binary Search is more suitable** because it provides **$O(\log n)$** search time, making product retrieval significantly faster than Linear Search's **$O(n)$** time complexity. Linear Search is only practical for small datasets or unsorted collections.